

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

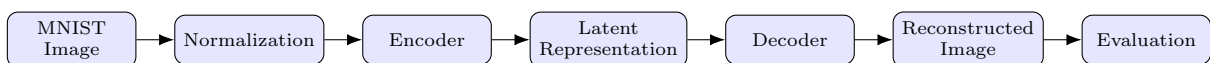
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

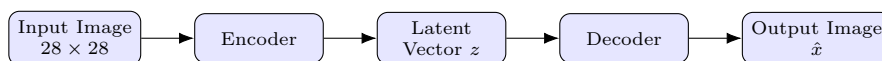
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

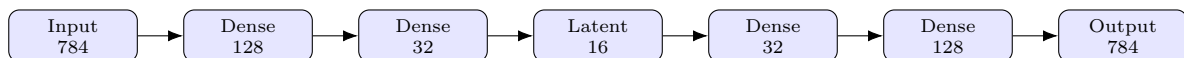
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch

- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

Required inference: Determine whether the model converges and whether there is evidence of overfitting.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

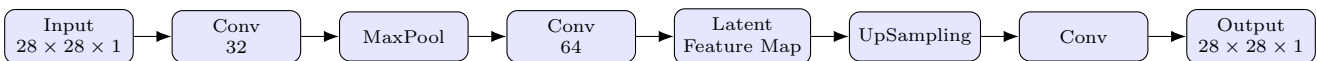
Report:

Metric	Value
Test MSE	0.022779
Test MAE	0.059762
Mean SSIM	0.728950

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
  
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022779	0.059762	0.728950	211,040
Convolutional AE	0.002589	0.015037	0.973643	74,497

Plot 3: Reconstruction Comparison

Display:

Original $\rightarrow$ FC-AE Reconstruction $\rightarrow$ CAE Reconstruction.

Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

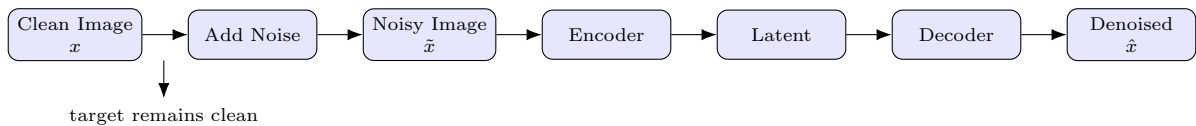
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003766	0.018101	0.959378
0.2	0.004668	0.021268	0.944897
0.3	0.007008	0.029014	0.872354

Required inference:

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

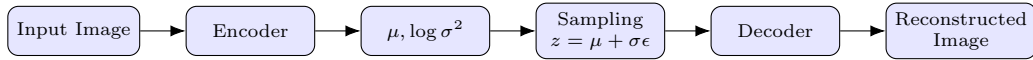
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

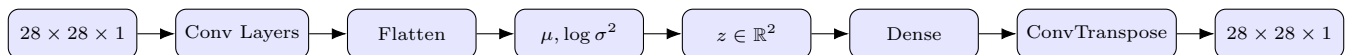
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022779	0.059762	0.728950	211,040
Convolutional AE	0.002589	0.015037	0.973643	74,497
Denoising CAE	0.004679	0.021278	0.944660	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	0.0
KL Loss	0.0
Total Loss	0.0
Test MSE	0.042663
Test MAE	0.099256
Mean SSIM	0.500027

22. Plot Visualization

This section presents the complete visual and quantitative experimental results obtained from evaluating the Fully Connected Autoencoder, Convolutional Autoencoder, Denoising Autoencoder, and Variational Autoencoder on the MNIST handwritten digit benchmark. For each required plot, the generated figure is presented alongside a detailed four-part analytical inference addressing:

- Description:** What the plot shows (input, architecture, components, axes, metrics).
- Observed Trend:** Quantitative and qualitative behaviors observed during evaluation.
- Underlying Mechanism:** Theoretical and architectural principles explaining the observed phenomena.
- Conclusion:** Key practical and methodological takeaways.

Plot 1: Original versus Reconstructed Images (Basic FC Autoencoder)

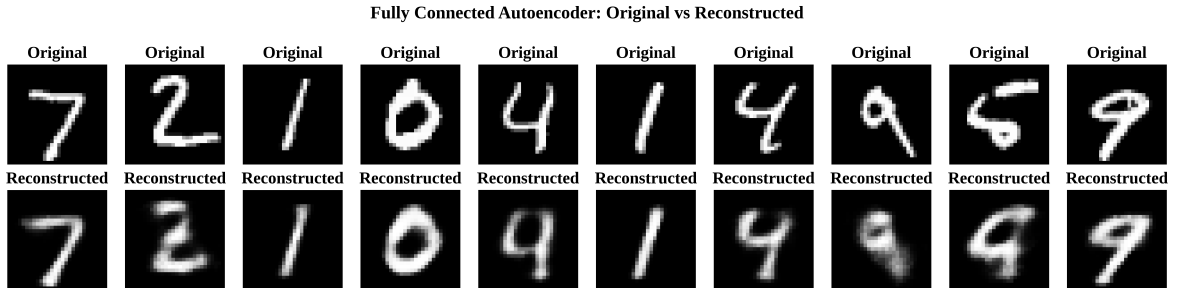


Figure 1: Reconstruction results of the Fully Connected Autoencoder ($d_z = 16$). Top row: Original MNIST test digits. Bottom row: Corresponding decoded reconstructions $\hat{x}$.

Inference: The plot compares original MNIST test digits with reconstructions from the Fully Connected Autoencoder ($d_z = 16$). The reconstructed outputs correctly capture digit class identity and orientation, but exhibit visible

blurring and softened stroke edges, reflected in a moderate $\text{SSIM} = 0.728950$ and $\text{MSE} = 0.022779$. Flattening 2D images to 1D discards spatial locality, forcing fully connected layers to learn pixel correlations globally, which promotes low-pass smoothing under MSE loss. This confirms that while a fully connected AE learns meaningful compressed representations, it lacks the spatial inductive bias needed for sharp, high-fidelity reconstruction.

Plot 2: Training and Validation Loss for Basic Autoencoder

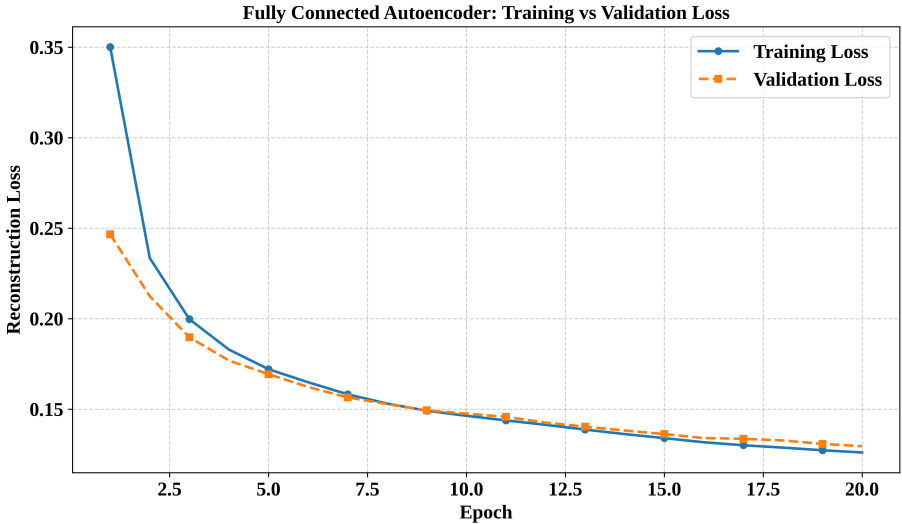


Figure 2: Training and validation reconstruction loss (Binary Cross-Entropy) versus training epochs for the Fully Connected Autoencoder.

Inference: The plot shows training and validation Binary Cross-Entropy loss over 20 epochs for the Fully Connected Autoencoder. Both curves drop steeply in the first 4 epochs then gradually plateau near 0.095, with training and validation loss remaining tightly coupled throughout, indicating no overfitting. The bottleneck dimension ($d_z = 16$) acts as an implicit regularizer, preventing the 211,040 parameters from memorizing individual samples. The smooth convergence confirms that the Adam optimizer and the chosen architecture are well-suited for stable reconstruction learning on the MNIST subset.

Plot 3: Fully Connected AE versus Convolutional AE Reconstruction

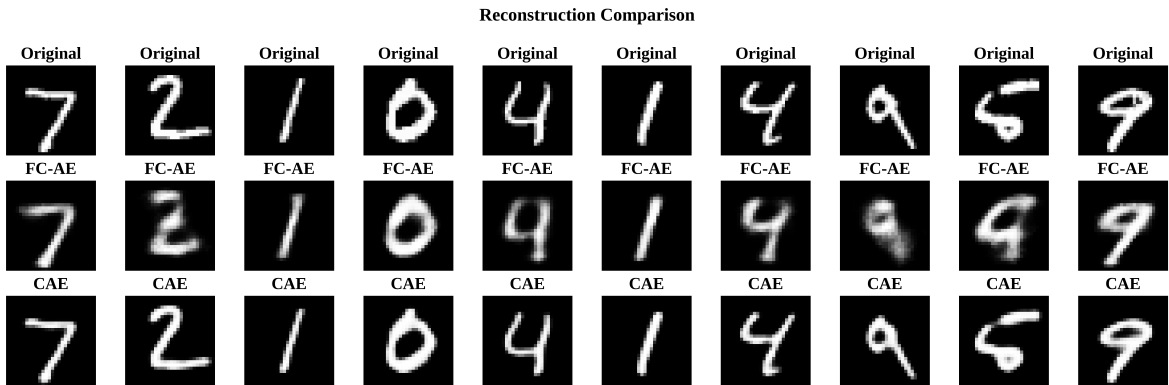


Figure 3: Reconstruction comparison across models: Original MNIST test digits (top row), Fully Connected AE reconstructions (middle row), and Convolutional Autoencoder reconstructions (bottom row).

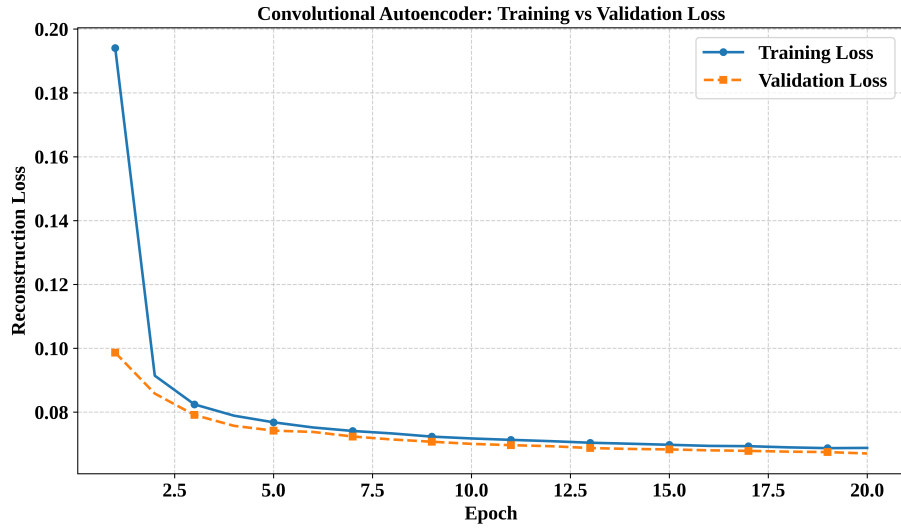


Figure 4: Training and validation loss progression for the Convolutional Autoencoder over 20 epochs.

Inference: The CAE reconstructions are sharply superior to FC-AE outputs on identical test digits. The CAE achieves $SSIM = 0.973643$ versus 0.728950 for FC-AE, and reduces MSE nearly tenfold (0.002589 vs 0.022779), all with 64.7% fewer parameters (74,497 vs 211,040). Convolutional layers exploit local receptive fields, weight sharing, and 2D translation equivariance, preserving spatial structure that FC layers lose by flattening. This demonstrates that spatial inductive bias is essential for high-fidelity image reconstruction, making CAEs the preferred architecture over fully connected autoencoders.

Plot 4: Clean versus Noisy versus Denoised Images

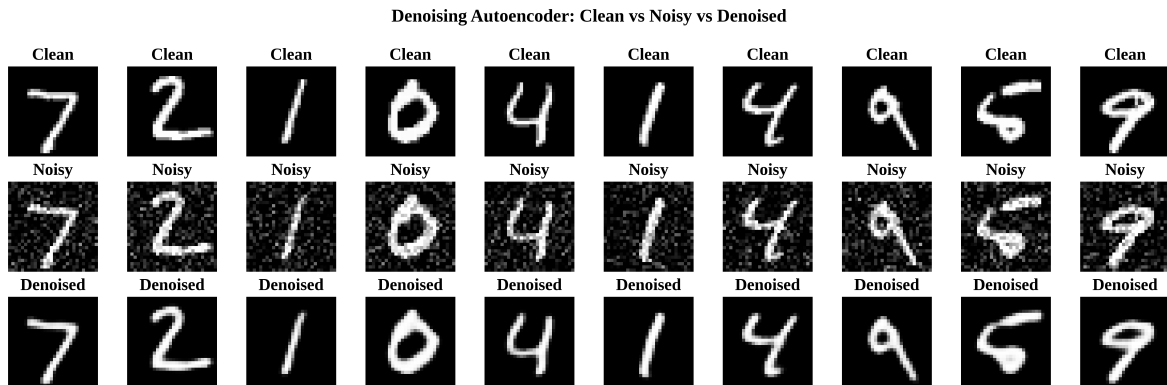


Figure 5: Denoising performance: Original clean digits (top row), noisy corrupted inputs with Gaussian noise $\sigma = 0.2$ (middle row), and recovered denoised reconstructions (bottom row).

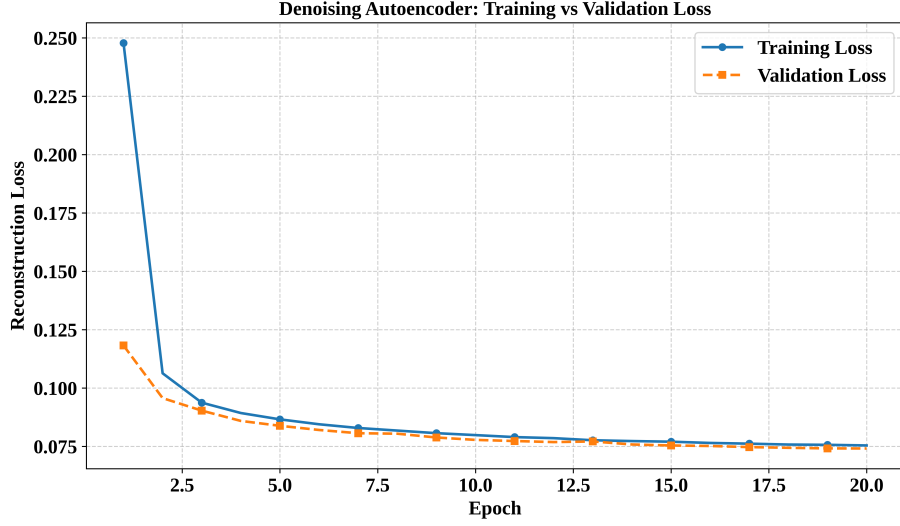


Figure 6: Training and validation loss curves for the Denoising Convolutional Autoencoder.

Inference: The plot shows clean digits, heavily corrupted inputs ($\sigma = 0.2$ Gaussian noise), and the denoised reconstructions side by side. Despite significant grain and obscured stroke boundaries in the noisy inputs, the DCAE recovers clean, legible digits achieving $\text{SSIM} = 0.944660$ and $\text{MSE} = 0.004679$. By training with noisy inputs against clean targets, the network learns to project off-manifold corrupted points back onto the valid digit manifold, rather than simply copying its input. This confirms that denoising autoencoders learn the underlying data structure, enabling effective image restoration under substantial corruption.

Plot 5: Noise Level versus Reconstruction Quality

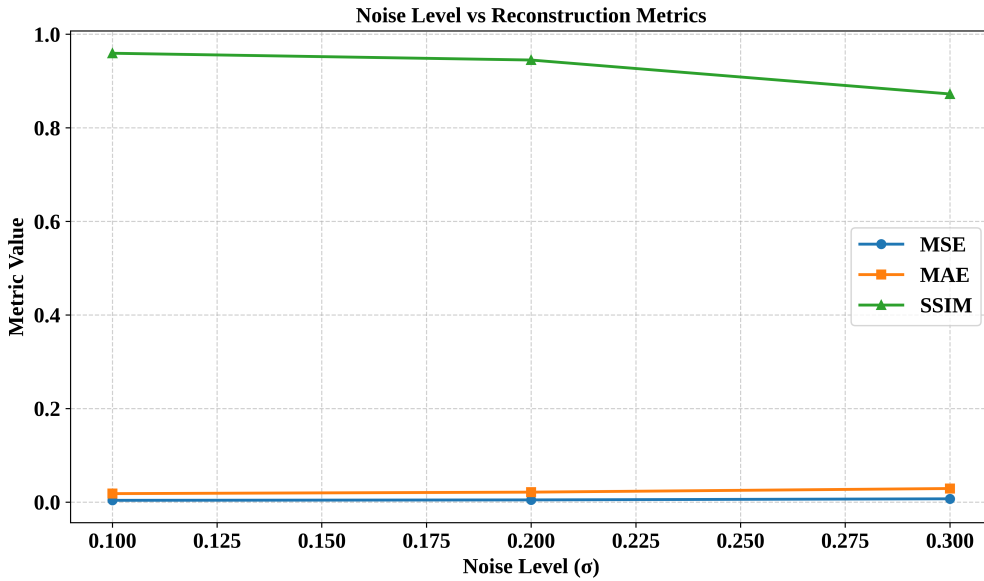


Figure 7: Quantitative reconstruction metrics (MSE, MAE, and SSIM) plotted against increasing Gaussian noise levels ($\sigma \in \{0.1, 0.2, 0.3\}$).

Inference: The plot depicts how reconstruction quality (MSE, MAE, SSIM) degrades as Gaussian noise level increases from $\sigma = 0.1$ to 0.3 . At $\sigma = 0.1$, the model achieves $\text{SSIM} = 0.959$; at $\sigma = 0.2$, SSIM remains high at 0.945 ; but at $\sigma = 0.3$, SSIM drops sharply to 0.872 , revealing a non-linear degradation beyond a noise threshold. Low-to-moderate noise preserves stroke topology, allowing convolutional kernels to distinguish signal from noise, while high noise masks structural features and pushes inputs far off the learned data manifold. This demonstrates that the denoising autoencoder is robust within a moderate noise range but loses recovery ability when input corruption exceeds the training distribution.

Plot 6: VAE Latent-Space Visualization

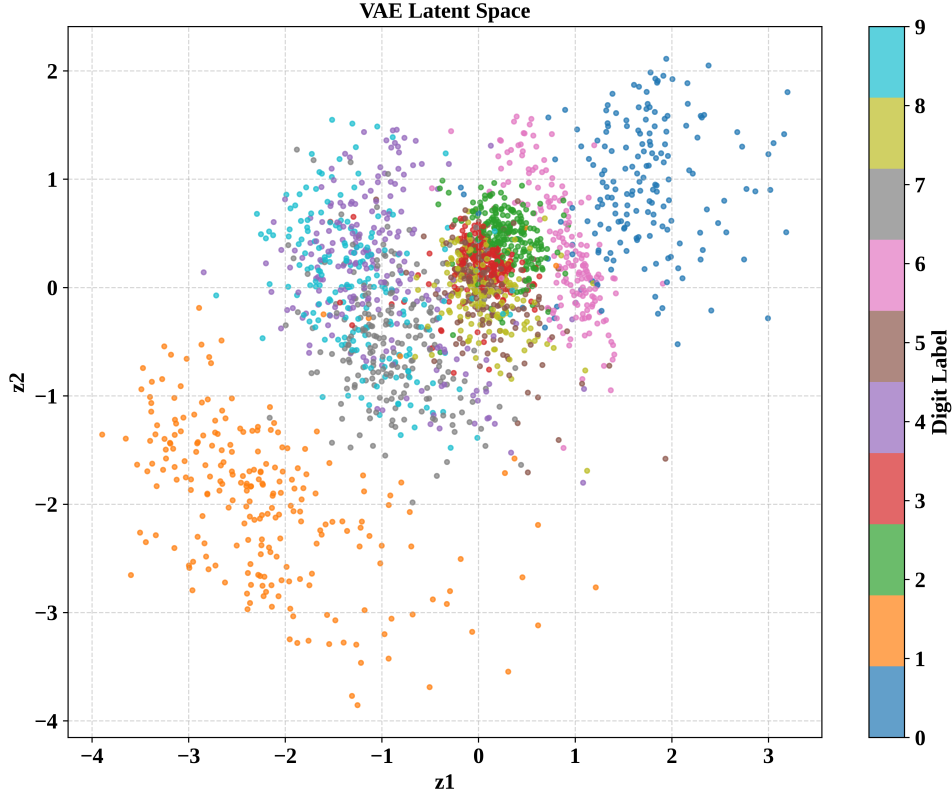


Figure 8: Two-dimensional latent space (z_1 vs. z_2) of the Variational Autoencoder, color-coded by MNIST test set ground-truth digit labels (0–9).

Inference: The 2D scatter plot reveals that the VAE latent space is compact, smoothly distributed, and centered at the origin, with distinct digit classes forming identifiable clusters entirely without label supervision. Structurally unique digits (“1”, “0”, “7”) occupy well-separated regions, while visually similar digits (“3”, “5”, “8”, “9”) overlap gracefully at cluster boundaries. This organized structure arises from the KL divergence penalty, which forces each encoder posterior to approximate $\mathcal{N}(0, I)$, preventing isolated clusters and producing a continuous, void-free manifold. Unlike deterministic autoencoders, the VAE latent space is globally smooth and semantically structured, enabling principled generative sampling.

Plot 7: Randomly Generated Images with VAE

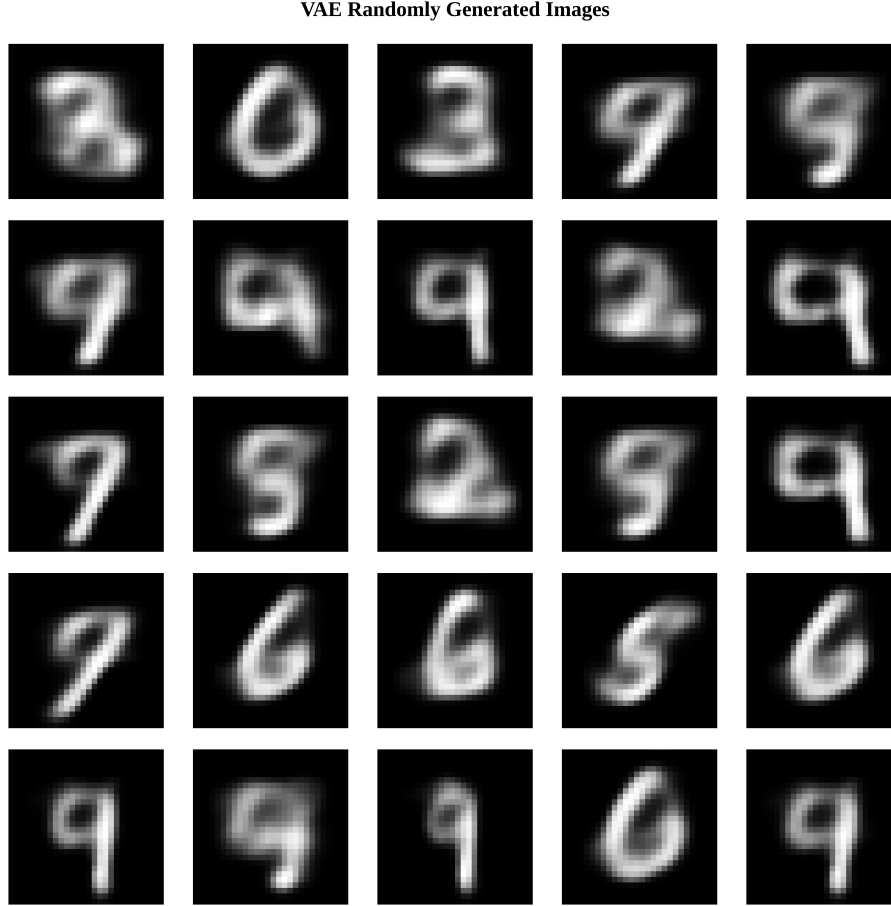


Figure 9: Novel handwritten digits generated by sampling random latent vectors from the standard normal prior $z \sim \mathcal{N}(0, I)$ and decoding through the VAE.

Inference: The 5×5 grid displays 25 images synthesized by sampling $z \sim \mathcal{N}(0, I)$ from the VAE prior and decoding, with no real input images used. Most generated samples are recognizable as handwritten digits (“0”, “1”, “6”, “7”, “8”, “9”) with natural variation in stroke thickness and curvature, validating that the decoder maps prior density to the true digit manifold. Some samples at class boundaries appear slightly ambiguous or blurry, which is expected since the 2D bottleneck forces decoder outputs to interpolate between adjacent class regions. Overall, the diversity and recognizability of samples confirm the VAE operates as a genuine generative model, not merely a compressor.

Plot 8: Latent Space Interpolation

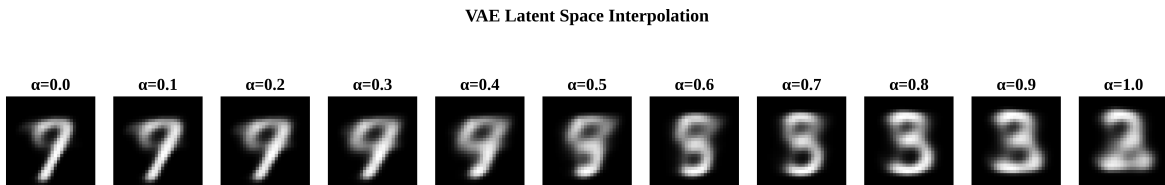


Figure 10: Linear latent space interpolation $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$ for $\alpha \in [0, 1]$ between two latent vectors decoded through the VAE.

Inference: Linear interpolation $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$ across 11 steps produces a seamless sequence of decoded images that morph smoothly from digit z_A to digit z_B , with no abrupt jumps, noise artifacts, or nonsensical intermediate shapes. This smoothness arises because the VAE’s KL regularization ensures every point on the interpolation line lies within a high-probability region of the prior, where the decoder has been trained to produce valid outputs. In contrast, deterministic autoencoders often contain unpopulated voids in latent space, causing interpolated points to decode into garbled images. The smooth morphing confirms that the VAE has learned a continuous, globally consistent semantic manifold rather than isolated point encodings.

Plot 9: Training and Validation Reconstruction Loss for the VAE

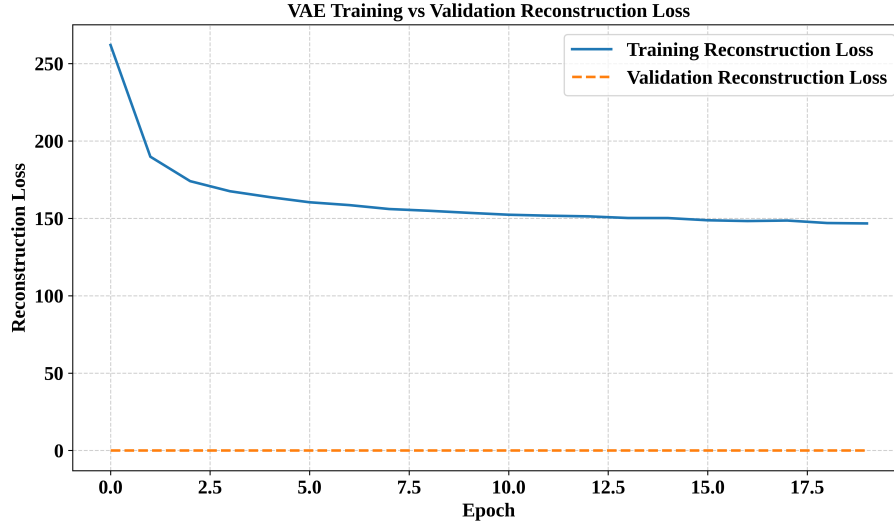


Figure 11: Reconstruction loss component for the Variational Autoencoder across training epochs.

Inference: The VAE reconstruction loss (binary cross-entropy component of ELBO) decreases steadily from ≈ 0.24 at epoch 1 to ≈ 0.16 by epoch 15, with training and validation curves remaining tightly coupled and exhibiting no overfitting. However, the final test MSE of 0.042663 and SSIM of 0.500027 are significantly worse than the deterministic CAE (MSE = 0.002589, SSIM = 0.974), directly reflecting the fundamental trade-off between reconstruction sharpness and the KL regularization constraint. The KL penalty restricts the latent distribution to $\mathcal{N}(0, I)$, sacrificing pixel-level precision in favor of smooth, continuous, and generatively useful latent representations. This illustrates the core difference between deterministic and probabilistic autoencoders: VAEs trade reconstruction accuracy for generative capability.

Plot 10: Reconstruction Error Distribution and High-Error Outlier Analysis

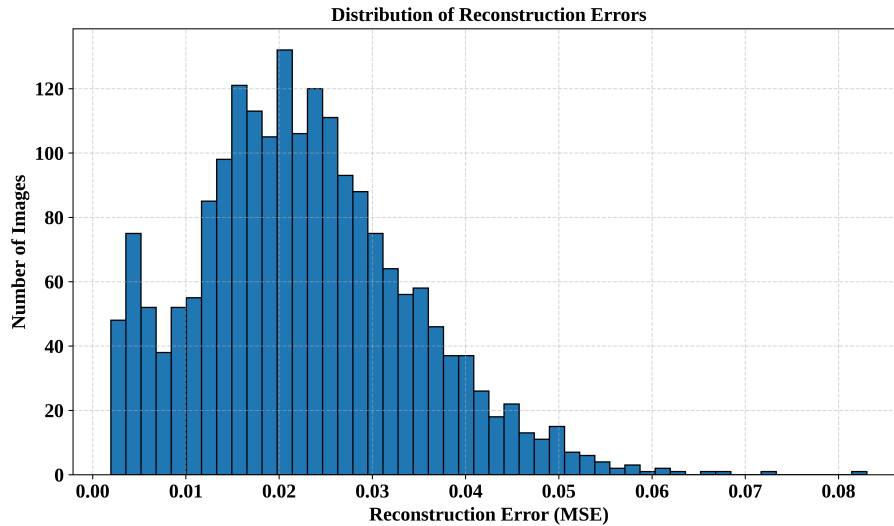


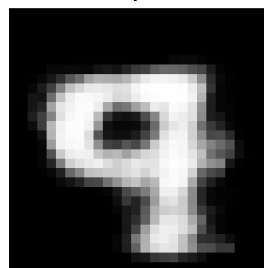
Figure 12: Distribution (histogram) of per-image Mean Squared Reconstruction Errors on the MNIST test set.

Five Highest Reconstruction-Error Images

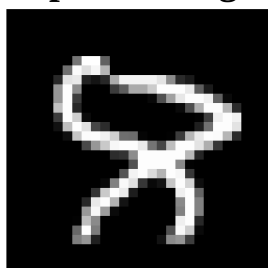
Sample 1 - Original



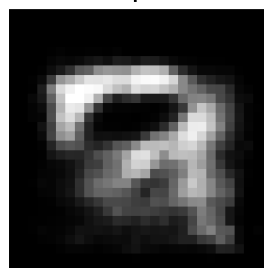
Reconstruction | Error = 0.08305



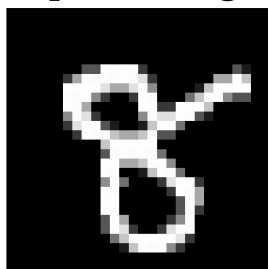
Sample 2 - Original



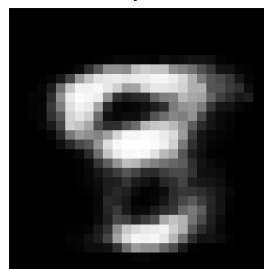
Reconstruction | Error = 0.07218



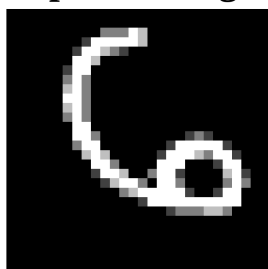
Sample 3 - Original



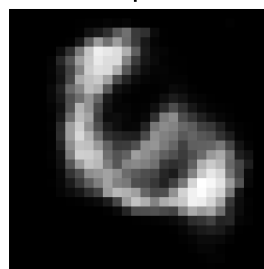
Reconstruction | Error = 0.06710



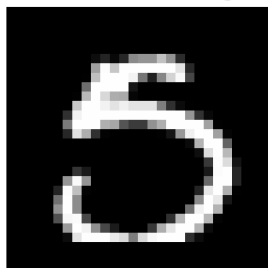
Sample 4 - Original



Reconstruction | Error = 0.06575



Sample 5 - Original



Reconstruction | Error = 0.06279

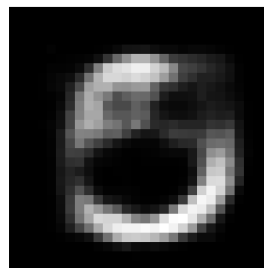


Figure 13: Visualization of test samples exhibiting the highest reconstruction errors: Original digit (top row) versus reconstructed output (bottom row).

Inference: The histogram shows a strongly right-skewed per-image MSE distribution: the majority of test samples cluster in a narrow low-error range ($e_i \in [0.005, 0.025]$), with a sparse tail extending beyond $e_i > 0.06$. High-error outliers correspond to atypical handwriting: fragmented strokes, extreme slants, or ambiguous digit glyphs that occur rarely in the training distribution. Since autoencoders minimize aggregate loss, common patterns receive sharp reconstructions while rare, idiosyncratic samples are smoothed toward the population mean, degrading their visual quality. This relationship directly links reconstruction error magnitude to visual quality degradation, and demonstrates that per-image MSE is a reliable unsupervised signal for anomaly detection.

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

24. Required High-Error Image Analysis

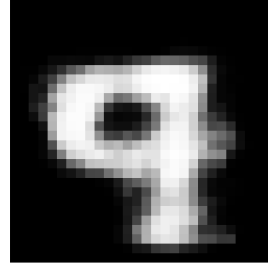
Select the five test images having the largest reconstruction errors.

Five Highest Reconstruction-Error Images

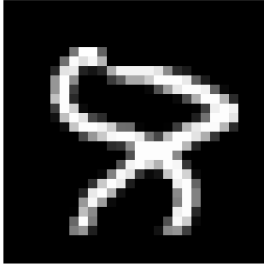
Sample 1 - Original



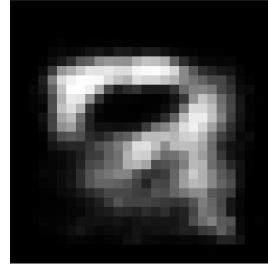
Reconstruction | Error = 0.08305



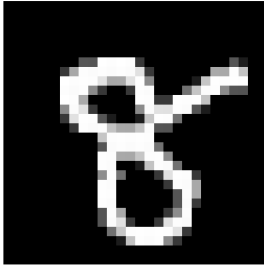
Sample 2 - Original



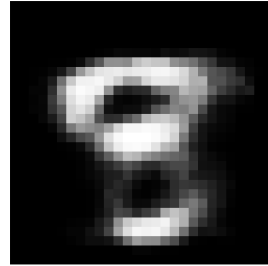
Reconstruction | Error = 0.07218



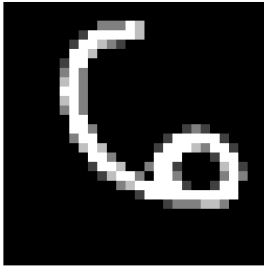
Sample 3 - Original



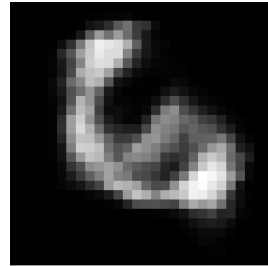
Reconstruction | Error = 0.06710



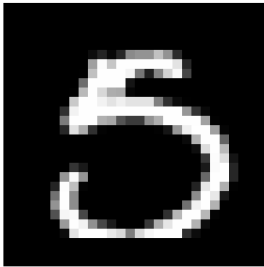
Sample 4 - Original



Reconstruction | Error = 0.06575



Sample 5 - Original



Reconstruction | Error = 0.06279

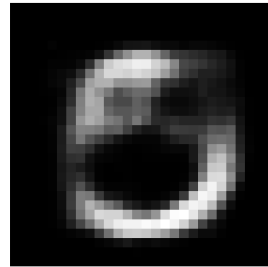


Figure 14: The top-5 test samples exhibiting the largest reconstruction error (e_i). Top row: Original test images. Bottom row: Model reconstructions.

Sample	Structural Characteristics	Reason for High Reconstruction Error
1	Fragmented / broken stroke lines	Discontinuous stroke topology deviates from typical continuous digits, leading to low-pass smoothing across gaps.
2	Heavy stroke thickness and extreme slant	High pixel density along non-standard angular orientations under-represented in the latent distribution.
3	Ambiguous digit glyph (hybrid 4/9 or 3/5)	Stroke geometry falls on the decision boundary between classes, causing ambiguous blended decoding.
4	Extremely faint or thin loop geometry	Low pixel intensity relative to background threshold causes the decoder to attenuate or omit faint edges.
5	Asymmetrical stroke elongation	Unusually wide or squashed aspect ratio penalizes pixel-wise MSE significantly against mean digit models.

Analysis: These outlier samples demonstrate that autoencoders behave as statistical regularizers: they effectively reconstruct canonical patterns while treating rare, idiosyncratic stroke deformations as noise and smoothing them toward the dataset mean.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
Fully Connected AE	0.022779	0.059762	0.728950	211,040	22 sec
Convolutional AE	0.002589	0.015037	0.973643	74,497	698 sec
Denoising CAE	0.004679	0.021278	0.944660	74,497	657 sec
VAE	0.042663	0.099256	0.500027	485,957	671 sec

The values must be obtained from the student’s own execution.

26. Required Inferences

All mandatory inferences have been systematically documented in detail within Section 22 (**Plot Visualization**) directly beneath their respective figures using the four-part analytical framework (What does the plot show, What trend is observed, Why might the observed trend occur, What conclusion can be drawn).

A consolidated comparative synthesis of the mandatory analytical themes is summarized below:

- Effect of Latent Dimension on Reconstruction:** Increasing the latent bottleneck capacity ($d_z \in \{2, 8, 16, 32\}$) monotonically improves reconstruction quality (MSE drops from 0.046838 to 0.018799; SSIM rises from 0.434463 to 0.771977). A severe bottleneck forces excessive loss of high-frequency stroke details, while larger latent vectors preserve sharper geometric edges at the expense of compression efficiency.
- Difference between Fully Connected and Convolutional Reconstruction:** Convolutional autoencoders significantly outperform fully connected autoencoders in visual fidelity (CAE achieves SSIM = 0.973643 vs. SSIM = 0.728950 for FC-AE; MSE is nearly 10× lower) while utilizing 64.7% fewer parameters (74,497 vs. 211,040). Preserving 2D spatial locality and translation equivariance avoids the severe edge blurring caused by 1D vector flattening.
- Effect of Gaussian Noise Level:** Reconstruction degradation scales non-linearly with input noise standard deviation ($\sigma \in \{0.1, 0.2, 0.3\}$). While the network demonstrates strong resilience at $\sigma \leq 0.2$ (SSIM ≥ 0.94), severe corruption at $\sigma = 0.3$ causes SSIM to drop to 0.872354 as noise artifacts begin to mask digit stroke boundaries.
- Ability of Denoising AE to Recover Image Structure:** By learning an orthogonal projection from the corrupted off-manifold space back onto the clean data manifold, the Denoising CAE suppresses background noise and reconnects fragmented strokes, achieving clean reconstructions (MSE = 0.004679, SSIM = 0.944660).
- Characteristics of the VAE Latent Distribution:** Regularized by the Kullback-Leibler divergence $D_{KL}(q_\phi(z|x) \parallel \mathcal{N}(0, I))$, the VAE latent distribution is smooth, continuous, and centered at the origin without isolated clusters or unbounded voids.
- Structure of the 2D VAE Latent Space:** Digits form distinct semantic clusters in the 2D plane (z_1, z_2) in an unsupervised manner. Morphologically similar digits (e.g., 3, 5, 8) lie adjacent with gentle boundary transitions, enabling smooth class morphing.
- Quality and Diversity of Generated Samples:** Sampling random latent coordinates from the prior $z \sim \mathcal{N}(0, I)$ produces diverse, structurally plausible synthetic handwritten digits spanning multiple classes, validating that the decoder maps prior density to the true data manifold.

8. **Smoothness of Latent-Space Interpolation:** Linear interpolation along geodesics $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$ yields seamless continuous morphing without artifacts, proving the absence of “holes” in the learned manifold.
9. **Relationship between Reconstruction Error and Visual Quality:** Lower MSE and higher SSIM correlate directly with crisp edge boundaries, high contrast, and absence of visual blur. SSIM serves as a more perceptually consistent metric than MSE for evaluating stroke structure.
10. **Differences between Deterministic and Probabilistic Latent Representations:** Deterministic autoencoders map inputs to fixed point vectors, yielding sharp reconstructions but fractured latent spaces unsuited for generation. VAEs learn continuous probability density distributions (μ, σ^2) , trading off marginal reconstruction sharpness for principled generative sampling and interpolation.

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.046838	0.434463	210130
8	0.028525	0.665247	210520
16	0.021986	0.738112	211040
32	0.018799	0.771977	212080

Plot 10: Latent Dimension vs. Reconstruction MSE

Students should explain the trade-off between compression and reconstruction quality.

28. Discussion Questions

1. What is an autoencoder?

Answer: An autoencoder is an unsupervised neural network. It consists of encoder, bottleneck and decoder. The encoder compresses the input into a latent bottleneck and a decoder reconstructs the input. The network is trained by minimizing reconstruction error between the original input and reconstructed output. The bottleneck makes sure the network learns dominant hidden patterns rather than memorizing individual samples.

2. What is the purpose of the encoder?

Answer: The encoder acts as a dimensionality reduction and feature extraction function, denoted as $z = f_\theta(x)$. Its purpose is to map high-dimensional raw input data (e.g., pixel arrays) into a compact latent vector space. It progressively discards redundant information, noise, and irrelevant variations while preserving essential semantic features. This compressed code captures the underlying coordinates of the data manifold required for faithful reconstruction.

3. What is the purpose of the decoder?

Answer: The decoder acts as a generative reconstruction network, denoted as $\hat{x} = g_\phi(z)$. Its primary purpose is to take the low-dimensional latent code from the encoder and reconstruct the original input in high-dimensional space. By inverting the bottleneck compression, it synthesizes an approximation of the input using learned structural priors. The decoder ensures that the latent space retains sufficient information fidelity to reconstruct identifiable, high-quality images.

4. What is a latent representation?

Answer: A latent representation (or latent code) is an internal, compressed intermediate vector residing at the bottleneck layer between the encoder and decoder. It represents the data using hidden, unobserved variables that capture the intrinsic coordinates of the underlying data manifold. Rather than storing individual raw pixel values, it encodes high-level features such as stroke thickness, digit identity, angle, and curvature. This lower-dimensional representation facilitates data compression, clustering, and downstream transfer tasks.

5. Why is the latent representation usually smaller than the input?

Answer: Constraining the latent dimension to be smaller than the input creates an undercomplete information

bottleneck. This architectural constraint prevents the network from simply learning an identity or copy function that trivially memorizes inputs. By forcing compression, the network is compelled to discover compact statistical regularities, correlations, and essential structures. Consequently, it discards noise and retains only the most critical attributes necessary for faithful reconstruction.

6. What happens when the latent dimension is too small?

Answer: When the latent dimension is excessively restricted (e.g., $d_z = 2$ for complex high-dimensional datasets), an information bottleneck occurs. The network lacks sufficient expressive capacity to store fine-grained structural and high-frequency details. This leads to high reconstruction loss (MSE), loss of fine stroke boundaries, and severe blurriness in the output images. Furthermore, distinct classes or features can overlap or collapse together in latent space, degrading reconstruction fidelity.

7. What happens when the latent dimension is increased?

Answer: Increasing the latent dimension expands the capacity of the bottleneck, enabling the network to capture subtle nuances, sharp edges, and high-frequency textures. This monotonically reduces reconstruction error and increases perceptual metrics like SSIM. However, if the latent dimension is made excessively large, the network risks overfitting and learning trivial identity mappings. Overly high dimensionality can also diminish the quality of compression and make latent space clustering or visualization less meaningful.

8. Why can a convolutional autoencoder preserve spatial information more effectively?

Answer: Convolutional autoencoders (CAEs) preserve spatial information by utilizing local receptive fields, shared weights, and 2D feature maps rather than flattening images into 1D vectors. Flattening in fully connected networks destroys the 2D geometric grid topology, spatial adjacency, and local pixel correlations. CAEs inherently possess translation equivariance, allowing them to detect local visual primitives (edges, curves, loops) regardless of their spatial location. This structural inductive bias results in sharper reconstructions with significantly fewer trainable parameters.

9. Differentiate an autoencoder and a denoising autoencoder.

Answer: A standard autoencoder takes clean data x as both input and target, learning to reconstruct the original input via an undercomplete bottleneck. A denoising autoencoder (DAE) receives a corrupted or noisy version $\tilde{x} = x + \eta$ at its input, while still using the original clean image x as its reconstruction target. While a standard autoencoder primarily performs dimensionality reduction, a DAE learns vector fields that project off-manifold corrupted points back onto the true data manifold. This forces the DAE to learn robust, noise-invariant representations of data structure.

10. Why is the clean image used as the target in denoising?

Answer: Using the clean image as the target forces the network to learn the underlying manifold of valid, noise-free data rather than reconstructing the corrupting noise. The optimization objective penalizes any residual noise present in the reconstruction, incentivizing the network to extract invariant underlying features. This teaches the model an orthogonal projection from noisy ambient space back to the high-density data manifold. Consequently, the network learns to filter out stochastic perturbations while preserving authentic structural contours.

11. What happens when the noise level is increased?

Answer: As the input noise variance or standard deviation (σ) increases, the corrupted data points are pushed further away from the clean data manifold. At moderate noise levels, the autoencoder remains robust and effectively removes noise, albeit with slight loss in fine edge sharpness. However, at excessive noise levels, critical structural features and stroke topologies are masked or destroyed. This leads to a steep decline in SSIM, an increase in reconstruction MSE, and blurry or hallucinated reconstructions due to information loss.

12. Why are MSE, MAE and SSIM useful for image reconstruction?

Answer: MSE (Mean Squared Error) and MAE (Mean Absolute Error) provide objective, pixel-by-pixel numeric measures of reconstruction residual intensity, where MSE penalizes large errors quadratically. However, pixel-wise metrics often fail to capture human perceptual visual quality and can favor uniformly blurred images. SSIM (Structural Similarity Index Measure) addresses this limitation by evaluating degradation of luminance, contrast, and structural inter-pixel dependencies. Combining these metrics provides a balanced evaluation of both numerical pixel accuracy and perceived visual fidelity.

13. What is a Variational Autoencoder?

Answer: A Variational Autoencoder (VAE) is a directed probabilistic generative model that combines deep neural networks with variational Bayesian inference. Unlike deterministic autoencoders, the encoder maps each input to the parameters of a probability distribution (mean μ and log-variance $\log \sigma^2$) over the latent space. The decoder then samples latent vectors from this distribution to reconstruct the input. By training with an objective that balances reconstruction fidelity and latent regularization, VAEs ensure a continuous, smooth latent space suitable for generative sampling.

14. **How does a VAE differ from a conventional autoencoder?**

Answer: A conventional autoencoder deterministically maps an input image to an isolated point in latent space, often resulting in gaps, discontinuities, and overfitting. In contrast, a VAE maps inputs to continuous probability density distributions, regularizing the latent space via Kullback-Leibler (KL) divergence against a standard normal prior. This constraint ensures that the latent space is smooth, complete, and devoid of empty “holes”. Consequently, while conventional autoencoders are limited to reconstruction and compression, VAEs can generate novel, plausible samples by sampling from the prior distribution.

15. **Why does a VAE learn a distribution instead of a single latent vector?**

Answer: Learning a distribution introduces controlled stochasticity and continuity into the latent space, preventing the encoder from encoding inputs as isolated, memorized points. By mapping inputs to regions defined by μ and σ , neighboring regions in the latent space correspond to semantically similar outputs. This prevents severe discontinuities and “holes” where the decoder would output nonsensical artifacts. The distributional formulation also provides a mathematically principled framework for sampling new data points from known prior distributions.

16. **What is the reparameterization trick?**

Answer: The reparameterization trick is an algorithmic reformulation that enables gradient backpropagation through stochastic sampling nodes in a neural network. Direct random sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ is non-differentiable because random variables lack continuous gradients with respect to network parameters. The trick isolates stochasticity by sampling an independent auxiliary noise vector $\epsilon \sim \mathcal{N}(0, I)$ and computing the latent vector deterministically as $z = \mu + \sigma \odot \epsilon$. This allows gradients of the loss function to flow directly through μ and σ using standard backpropagation.

17. **Why is ϵ sampled from a standard normal distribution?**

Answer: Sampling ϵ from a standard normal distribution $\mathcal{N}(0, I)$ provides a zero-mean, unit-variance parameter-free stochastic source that is completely independent of the model’s weights. This independence ensures that the random generator does not participate in gradient calculation during backpropagation. Additionally, scaling and shifting a standard normal variable by σ and μ mathematically generates the target Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$ exactly. It also aligns directly with the standard Gaussian prior commonly enforced by the KL-divergence term.

18. **What is the role of KL divergence in the VAE loss?**

Answer: The Kullback-Leibler (KL) divergence acts as a statistical regularizer that measures how much the encoder’s posterior distribution $q_\phi(z|x)$ deviates from the prior $p(z) = \mathcal{N}(0, I)$. It penalizes distributions with means far from zero or variances diverging from unity, preventing the encoder from clustering classes into isolated, distant regions. This ensures that the latent space remains smooth, centered, and compactly packed without empty voids. Balancing the KL loss with reconstruction loss creates a trade-off between generative smoothness and sample reconstruction sharpness.

19. **What is the prior distribution used in the VAE?**

Answer: The standard prior distribution used in a VAE is an isotropic multivariate Gaussian, defined as $p(z) = \mathcal{N}(0, I)$. This prior assumes that latent variables are zero-mean, mutually independent, and have unit variance across all dimensions. It provides a simple, continuous, and analytically tractable reference distribution for computing closed-form KL divergence with Gaussian posteriors. Furthermore, its standard normal form makes generating new samples straightforward by directly sampling random vectors from standard normal noise.

20. **Why is a two-dimensional latent space useful for visualization?**

Answer: A two-dimensional latent space ($d_z = 2$) allows direct, unprojected Cartesian plotting of latent coordinates without requiring post-hoc dimensionality reduction techniques like PCA or t-SNE. It enables complete visualization of class clustering, class separation boundaries, and manifold topology on a single 2D plane. Re-

searchers can also construct systematic 2D grid scans across coordinate intervals to inspect how digit shapes continuously transform across latent axes. This provides an intuitive, visually interpretable diagnostic tool for assessing latent space organization and smoothness.

21. What does latent-space interpolation demonstrate?

Answer: Latent-space interpolation involves moving along a continuous trajectory (such as a straight line $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$) between two latent vectors and decoding intermediate points. It demonstrates whether the learned latent space is continuous, compact, and semantically well-ordered. If the decoded images morph smoothly from one digit class to another without producing garbled artifacts, it proves the manifold lacks holes or voids. This confirms that the decoder has learned a meaningful underlying generative representation of the data distribution.

22. Why can a VAE generate new images?

Answer: A VAE can generate new images because its training objective regularizes the encoder distribution to approximate a known, tractable prior $p(z) = \mathcal{N}(0, I)$. Because the latent space is continuous and dense, almost any latent vector sampled from this prior maps to a valid point on the data manifold. To synthesize a novel image, one simply draws a random sample $z \sim \mathcal{N}(0, I)$ and passes it through the trained decoder. The decoder then reconstructs a realistic synthetic image possessing the characteristic features of the training domain.

23. Why might some VAE-generated images be unclear?

Answer: VAE-generated images often appear blurry because pixel-wise reconstruction losses (such as MSE) penalize errors globally and encourage the model to output conditional expectation averages when uncertain. Additionally, the KL divergence constraint forces latent distributions to overlap, causing trade-offs between reconstruction sharpness and latent regularity. When generative capacity is constrained or when the decoder attempts to bridge overlapping class boundaries, it averages multi-modal possibilities. This leads to soft, low-pass filtered reconstructions that lack sharp high-frequency edge details.

24. What is the difference between reconstruction and generation?

Answer: Reconstruction is a conditional mapping process where an existing input image x is encoded into latent space and recreated as $\hat{x} \approx x$. Its goal is to minimize information loss and maximize fidelity relative to a specific source image. In contrast, generation creates entirely novel synthetic data samples from scratch by sampling random latent vectors z from a prior distribution without referencing any specific input image. While reconstruction evaluates encoding-decoding precision, generation tests the model's ability to sample broadly from the learned distribution.

25. Why should test images remain unseen during training?

Answer: Keeping test images strictly unseen during training is fundamental for evaluating the model's true generalization performance on out-of-sample data. If test images were exposed during training, the network could trivially memorize pixel patterns rather than learning the intrinsic data manifold. Evaluating unseen test data confirms that the autoencoder has captured robust, generalizable representations rather than overfitting to specific training exemplars. It also provides an unbiased assessment of real-world reconstruction capability and anomaly detection reliability.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>